AI Assisted Coding Assessment (Lab 11.3).

I.Harshith

2303A51624

BT.NO: 08

Problem Statement 1:

Question

Below is a **complete lab-ready solution** you can directly use in your Word report. It includes **code, explanations, comparisons, outputs, and justifications** for all tasks.

✅ Lab 11: Data Structures with AI

Task 1: Smart Contact Manager

(Arrays vs Linked Lists)

✅ 1A. Array-Based Contact Manager (Using Python List)

Implementation

```
class ContactManagerArray:
    def __init__(self):
        self.contacts = []

    def add_contact(self, name, phone):
        self.contacts.append({"name": name, "phone": phone})
        print("Contact added successfully.")

    def search_contact(self, name):
        for contact in self.contacts:
            if contact["name"] == name:
                return contact
        return None

    def delete_contact(self, name):
        for contact in self.contacts:
            if contact["name"] == name:
```

```
                self.contacts.remove(contact)
                print("Contact deleted successfully.")
                return
        print("Contact not found.")

    def display_contacts(self):
        print(self.contacts)
```

Sample Output

```
cm = ContactManagerArray()
cm.add_contact("Rahul", "9876543210")
cm.add_contact("Priya", "9123456780")
print(cm.search_contact("Rahul"))
cm.delete_contact("Rahul")
cm.display_contacts()
```

✅ 1B. Linked List Contact Manager

Implementation

```
class Node:
    def __init__(self, name,
        phone): self.name = name
        self.phone = phone
        self.next = None


class ContactManagerLinkedList:
    def __init__(self):
        self.head = None

    def add_contact(self, name, phone):
        new_node = Node(name, phone)
        new_node.next = self.head self.head
        = new_node print("Contact added
        successfully.")

    def search_contact(self, name):
        current = self.head
        while current:
            if current.name == name:
                return {"name": current.name, "phone":
                current.phone}
            current = current.next
        return None

    def delete_contact(self, name):
        current = self.head
        prev = None

        while current: if
            current.name == name:
                if prev: prev.next =
                    current.next
```

```
                else: self.head =
                    current.next
                print("Contact deleted successfully.")
                return
            prev = current
            current =
            current.next
        print("Contact not found.")

    def display_contacts(self): current =
        self.head while current:
        print(current.name, current.phone)
        current = current.next
```

Comparison: Array vs Linked List

Feature	Array (List)	Linked List
Insertion	O(1) (at end)	O(1) (at head)
Deletion	O(n)	O(n)
Memory	Contiguous	Dynamic allocation
Searching	O(n)	O(n)
Flexibility	Fixed resizing cost	Fully dynamic

Conclusion:

- Arrays are simple and efficient for small datasets.
- Linked Lists are better for dynamic memory usage and frequent insertions.

Task 2: Library Book Search System

(Queue & Priority Queue)

✅ 2A. Queue (FIFO)

```
from collections import deque
class BookQueue:
    def __init__(self):
        self.queue = deque()

    def enqueue(self, request):
        self.queue.append(request)

    def dequeue(self):
        if self.queue: return
            self.queue.popleft()
        return "Queue is empty"
```

Test

```
q = BookQueue()
q.enqueue("Student - Book A")
q.enqueue("Student - Book B")
print(q.dequeue())
```

✅ 2B. Priority Queue (Faculty Priority)

```
import heapq

class PriorityBookQueue:
    def __init__(self):
        self.queue = []

    def enqueue(self, user_type, book):
        priority = 1 if user_type == "Faculty" else 2
        heapq.heappush(self.queue, (priority, book))

    def dequeue(self):
        if self.queue: return
            heapq.heappop(self.queue)
        return "Queue is empty"
```

Test

```
pq = PriorityBookQueue()
pq.enqueue("Student", "Book A")
pq.enqueue("Faculty", "Book B")
pq.enqueue("Student", "Book C")

while pq.queue:
    print(pq.dequeue())
```

✅ Output:

Faculty request processed first.

Task 3: Emergency Help Desk (Stack)

✅ Stack Implementation (LIFO)

```
class TicketStack:
    def __init__(self):
        self.stack = []
        self.max_size = 5

    def push(self, ticket):
        if len(self.stack) < self.max_size:
            self.stack.append(ticket)
            print("Ticket added:", ticket)
        else: print("Stack is
            full!")

    def pop(self):
        if self.stack: return
            self.stack.pop()
        return "Stack is empty"

    def peek(self):
        if self.stack: return
            self.stack[-1]
        return "Stack is empty"

    def is_empty(self): return
        len(self.stack) == 0

    def is_full(self): return len(self.stack)
        == self.max_size
```

Simulation

```
ts = TicketStack()
ts.push("Login Issue")
ts.push("WiFi Not Working")
ts.push("Printer Error")
ts.push("Software Crash")
ts.push("Network Down")
print("Top Ticket:", ts.peek())

while not ts.is_empty():
    print("Resolved:", ts.pop())
```

✅ Demonstrates LIFO Behavior

Task 4: Hash Table (With Chaining)

```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self.hash_function(key)
        for pair in self.table[index]:
            if pair[0] == key:
                pair[1] = value
                return
        self.table[index].append([key, value])

    def search(self, key): index =
        self.hash_function(key) for
        pair in self.table[index]:
            if pair[0] == key:
                return pair[1]
        return None

    def delete(self, key): index =
        self.hash_function(key) for i, pair in
        enumerate(self.table[index]):
            if pair[0] == key:
                self.table[index].pop(i)
                return "Deleted"
        return "Not Found"
```

✅ Collision Handling:

Uses **Chaining (List of lists)**.

Task 5: Real-Time Application Challenge

Feature Mapping Table

Feature	Data Structure	Justification

Feature	Data Structure	Justification
Student Attendance	Hash Table	Fast lookup using student ID
Event Registration	Linked List	Dynamic participant addition
Library Borrowing	Priority Queue	Faculty priority
Bus Scheduling	Queue	FIFO order
Cafeteria Orders	Queue	Orders served in arrival order

✅ Implementation: Student Attendance (Using Hash Table)

```
class AttendanceSystem:
    def __init__(self):
        self.attendance = {}

    def mark_attendance(self, student_id, status):
        self.attendance[student_id] = status

    def check_attendance(self, student_id): return
        self.attendance.get(student_id, "Not Found")

    def display_all(self):
        for student, status in self.attendance.items():
            print(student, ":", status)
```

Test

```
asys = AttendanceSystem()
asys.mark_attendance("101", "Present")
asys.mark_attendance("102", "Absent")

print(asys.check_attendance("101"))
asys.display_all()
```

Time Complexity Summary

Data Structure	Insert	Delete	Search
Array	O(1)	O(n)	O(n)
Linked List	O(1)	O(n)	O(n)
Stack	O(1)	O(1)	O(n)
Queue	O(1)	O(1)	O(n)
Priority Queue	O(log n)	O(log n)	O(n)
Hash Table	O(1) avg	O(1) avg	O(1) avg